



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

EXPERIMENT :6

Student Name: Piyush Patel

UID: 23BCS12673

Branch: BE-CSE

Section/Group: KRG-1A

Date of Performance: 25/09/2025

Semester: V

Subject Name: ADBMS

Subject Code: 23CSP-333

1.Aim

i)Stored Procedures: HR-Analytics: Employee count based on dynamic gender passing (Medium)

TechSphere Solutions, a growing IT services company with offices across India, wants to track and monitor gender diversity within its workforce. The HR department frequently needs to know the total number of employees by gender (Male or Female). To solve this problem, the company needs an automated database-driven solution that can instantly return the count of employees by gender through a stored procedure that:

1. Create a PostgreSQL stored procedure that:
2. Takes a gender (e.g., 'Male' or 'Female') as input.
3. Calculates the total count of employees for that gender.
4. Returns the result as an output parameter.
5. Displays the result clearly for HR reporting purposes.

ii) Stored Procedures: SmartStore Automated Purchase System (Hard)

SmartShop is a modern retail company that sells electronic gadgets like smartphones, tablets, and laptops. The company wants to automate its ordering and inventory management process. Whenever a customer places an order, the system must:

1. Verify stock availability for the requested product and quantity.
2. If sufficient stock is available:
 - Log the order in the sales table with the ordered quantity and total price.
 - Update the inventory in the products table by reducing quantity_remaining and increasing quantity_sold.
 - Display a real-time confirmation message: "Product sold successfully!"
3. If there is insufficient stock, the system must:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

- Reject the transaction and display: Insufficient Quantity Available!"

2. Tools Used: SQL Server Management Studio

3. Query

Q1)

```
CREATE TABLE employee_info (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    gender VARCHAR(10) NOT NULL,  
    salary NUMERIC(10,2) NOT NULL,  
    city VARCHAR(50) NOT NULL  
);  
  
INSERT INTO employee_info (name, gender, salary, city)  
VALUES  
  
('Alok', 'Male', 50000.00, 'Delhi'),  
('Priya', 'Male', 60000.00, 'Mumbai'),  
('Rajesh', 'Female', 45000.00, 'Bangalore'),  
('Sneha', 'Male', 55000.00, 'Chennai'),  
('Anil', 'Male', 52000.00, 'Hyderabad'),  
('Sunita', 'Female', 48000.00, 'Kolkata'),  
('Vijay', 'Male', 47000.00, 'Pune'),  
('Ritu', 'Male', 62000.00, 'Ahmedabad'),  
('Amit', 'Female', 51000.00, 'Jaipur');  
  
CREATE OR REPLACE PROCEDURE sp_get_employees_by_gender(  
    IN p_gender VARCHAR(50),  
    OUT p_employee_count INT  
)  
  
LANGUAGE plpgsql  
  
AS $$
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.
BEGIN

```
SELECT COUNT(id)
INTO p_employee_count
FROM employee_info
WHERE gender = p_gender;
```

```
RAISE NOTICE 'Total employees with gender %: %', p_gender, p_employee_count;
END;
$$;
CALL sp_get_employees_by_gender('Male', NULL);
```

Q2)

```
CREATE TABLE products (
    product_code VARCHAR(10) PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) NOT NULL,
    quantity_remaining INT NOT NULL,
    quantity_sold INT DEFAULT 0
);

CREATE TABLE sales (
    order_id SERIAL PRIMARY KEY,
    order_date DATE NOT NULL,
    product_code VARCHAR(10) NOT NULL,
    quantity_ordered INT NOT NULL,
    sale_price NUMERIC(10,2) NOT NULL,
    FOREIGN KEY (product_code) REFERENCES products(product_code)
);

INSERT INTO products (product_code, product_name, price, quantity_remaining, quantity_sold)
VALUES
('P001', 'iPhone 13 Pro Max', 109999.00, 10, 0),
('P002', 'Samsung Galaxy S23 Ultra', 99999.00, 8, 0),
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
('P003', 'iPad AIR', 55999.00, 5, 0),
('P004', 'MacBook Pro 14"', 189999.00, 3, 0),
('P005', 'Sony WH-1000XM5 Headphones', 29999.00, 15, 0);
INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
VALUES
('2025-09-15', 'P001', 1, 109999.00),
('2025-09-16', 'P002', 2, 199998.00),
('2025-09-17', 'P003', 1, 55999.00),
('2025-09-18', 'P005', 2, 59998.00),
('2025-09-19', 'P001', 1, 109999.00);
SELECT * FROM PRODUCTS;
SELECT * FROM SALES;
```

```
CREATE OR REPLACE PROCEDURE pr_buy_products(
    IN p_product_name VARCHAR,
    IN p_quantity INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_product_code VARCHAR(20);
    v_price FLOAT;
    v_count INT;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM products
    WHERE product_name = p_product_name
    AND quantity_remaining >= p_quantity;
```

```

IF v_count > 0 THEN
    SELECT product_code, price
    INTO v_product_code, v_price
    FROM products
    WHERE product_name = p_product_name;
    INSERT INTO sales (order_date, product_code, quantity_ordered, sale_price)
    VALUES (CURRENT_DATE, v_product_code, p_quantity, (v_price * p_quantity));
    UPDATE products
    SET quantity_remaining = quantity_remaining - p_quantity,
        quantity_sold = quantity_sold + p_quantity
    WHERE product_code = v_product_code;
    RAISE NOTICE 'PRODUCT SOLD..! Order placed successfully for % unit(s) of %.',
    p_quantity, p_product_name;
ELSE
    RAISE NOTICE 'INSUFFICIENT QUANTITY..! Order cannot be processed for % unit(s) of
    %.', p_quantity, p_product_name;
END IF;
END;
$$;
CALL pr_buy_products ('MacBook Pro 14"', 1);

```

4.Outputs

✓ Successfully run. Total query runtime: 57 msec. 1 rows affected. ✕

Total rows: 1 Query complete 00:00:00.057 CRLF Ln 40, Col 1

	product_code [PK] character varying (10)	product_name character varying (100)	price numeric (10,2)	quantity_remaining integer	quantity_sold integer
1	P001	IPHONE 13 PRO MAX	109999.00	10	0
2	P002	Samsung Galaxy S23 Ultra	99999.00	8	0
3	P003	IPAD AIR	55999.00	5	0
4	P004	MacBook Pro 14"	189999.00	3	0
5	P005	Sony WH-1000XM5 Headphones	29999.00	15	0



5. Learning Outcomes

- Learned how to create and execute stored using plpgsql language for handling business logic inside the database.
- Developed the ability to implement conditional checks and validations inside procedures.
- Learned how to combine INSERT, UPDATE, and SELECT queries within stored procedures to automate tasks.